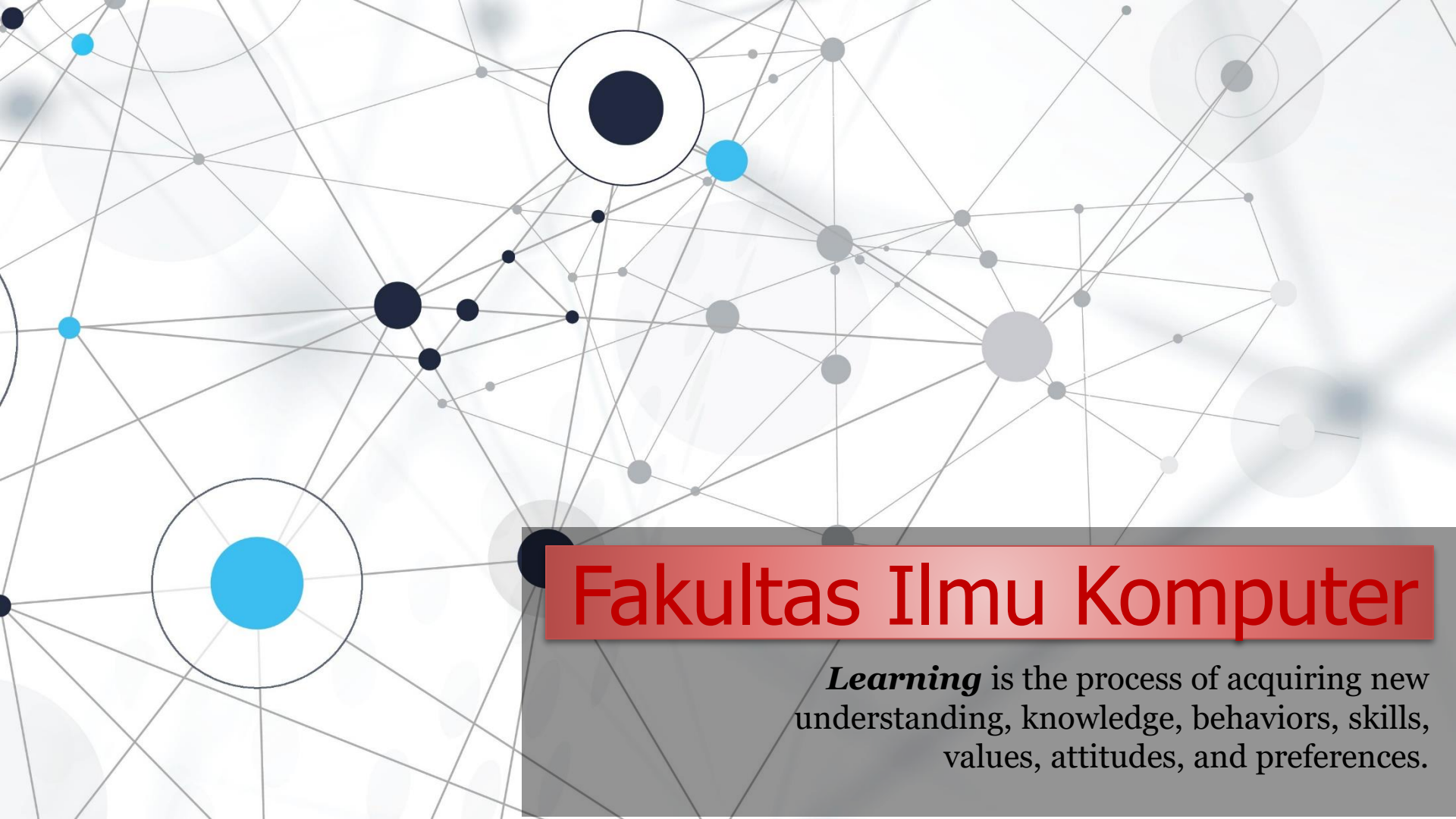


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

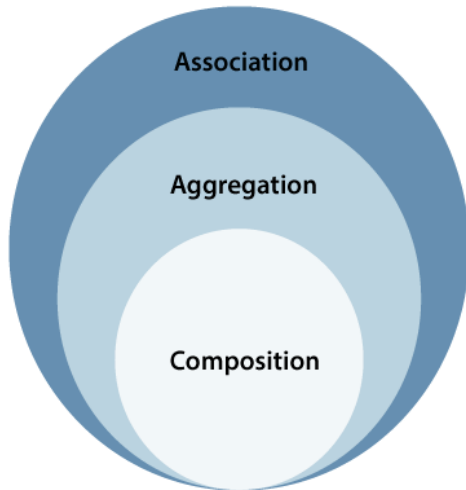
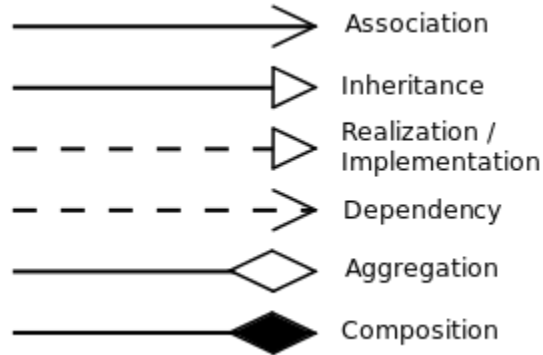
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Agrégation Concept *in* Python OOP



**Welcome to *the* Object Oriented
Programming *course***

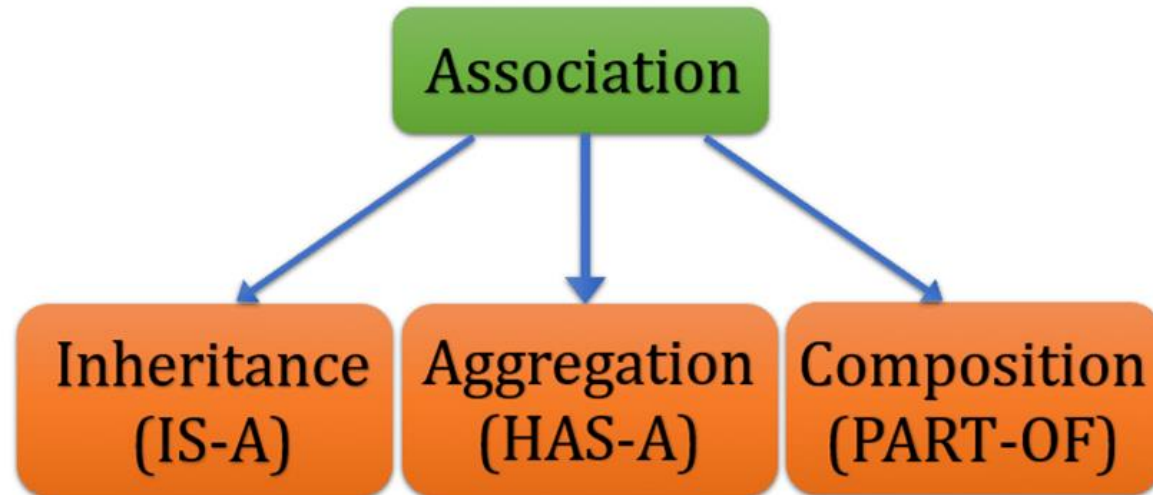
Types of Associations in OOP



- 1) Asosiasi biasa (*ordinary association*): objek memiliki hubungan yang terdefinisi secara jelas dan objek tersebut tidak bergantung satu sama lain.
- 2) **Agregasi (*aggregation*)**: satu objek memiliki beberapa objek terkait, tetapi objek-objek tersebut dapat ada tanpa objek utama.
- 3) **Komposisi (*composition*)**: satu objek terdiri dari beberapa objek terkait, dan objek-objek tersebut hanya dapat ada dengan objek utama.
- 4) Inheritance (*pewarisan*): objek turunan mewarisi sifat dan perilaku dari objek induk.

Penggunaan asosiasi yang tepat dapat membantu mengorganisir kode program dan membuat program lebih mudah dipahami dan dikembangkan.

Aggregation in OOP



Association adalah hubungan antara dua objek yang menggambarkan bahwa satu objek terkait dengan objek lain. Objek-objek ini memiliki hubungan terpisah, dan keduanya dapat bertahan hidup tanpa satu sama lain.

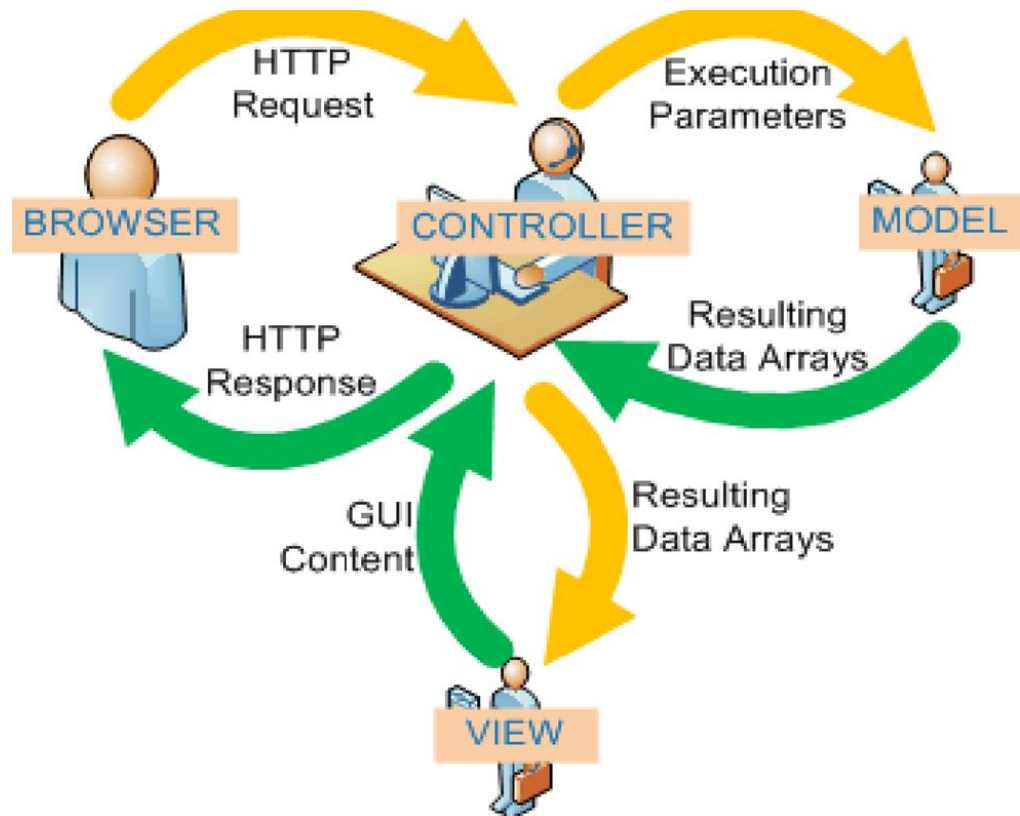
Aggregation adalah jenis hubungan di mana satu objek berfungsi sebagai pemilik objek lainnya, dan objek-objek ini membentuk suatu kelompok. Objek yang dimiliki tidaklah menjadi milik objek pemilik, dan dapat dimiliki oleh objek lain.

Composition merupakan jenis hubungan yang serupa dengan Aggregation. Namun, perbedaannya adalah objek yang dimiliki tidak dapat berfungsi tanpa objek pemiliknya. Dalam komposisi, objek yang dimiliki menjadi milik objek pemilik, dan objek tersebut hanya dapat dimiliki oleh satu objek pemilik pada satu waktu.

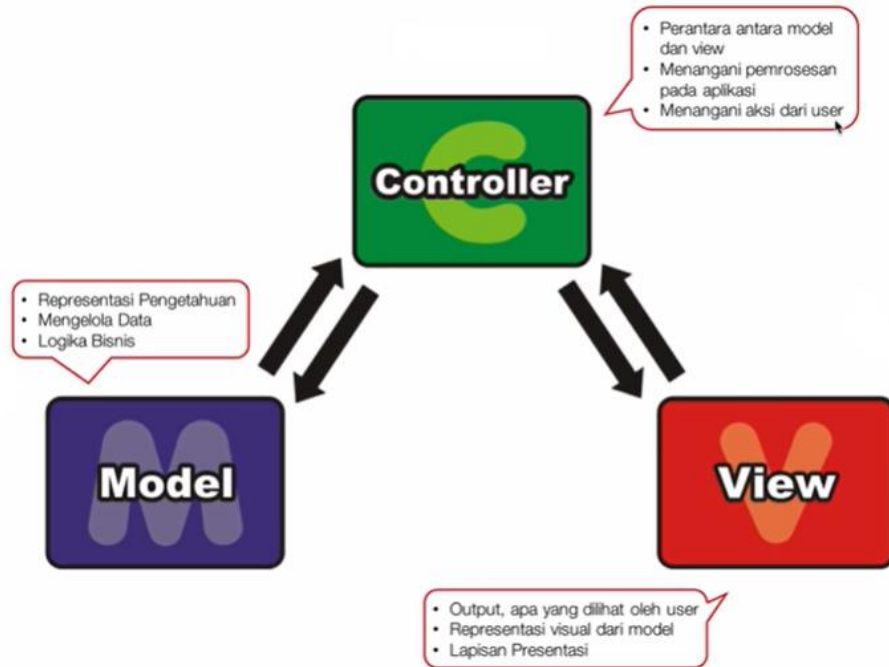
Types of Associations in OOP

Association, aggregation dan composition adalah tiga konsep penting dalam pemrograman berorientasi objek (OOP) yang digunakan untuk **menggambarkan hubungan antara objek di dalam sistem.**
Hubungan antara satu class dengan class yang lain.

MVC Framework

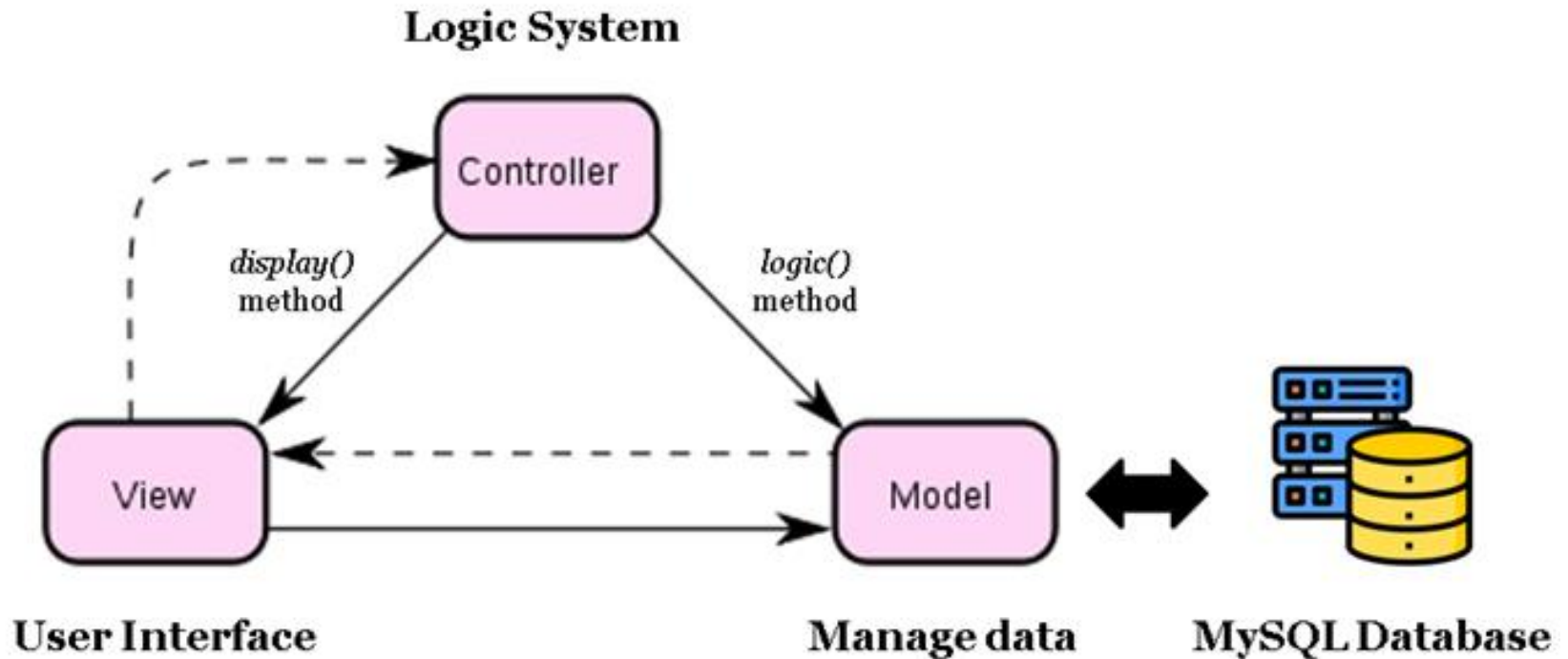


MVC Framework



- ❖ MVC (Model-View-Controller) is a concept used in software development that is separated between three main components, namely Model, View, and Controller.
- ❖ This concept can be implemented in an object-oriented approach (OOP) to simplify the development of complex software and facilitate the maintenance process.

- ✓ Dalam MVC, **Model adalah bagian dari aplikasi yang mengatur data dan bisnis logic**, **View adalah bagian dari aplikasi yang mengatur presentasi dan tampilan antarmuka pengguna**, sedangkan **Controller adalah bagian dari aplikasi yang mengatur interaksi antara View dan Model**.
- ✓ Contoh framework berbasis MVC dalam pendekatan OOP adalah **Laravel**, yang merupakan framework PHP open-source yang populer dan banyak digunakan oleh developer web. **Laravel** memungkinkan pengembangan aplikasi web yang mudah, cepat, dan efisien dengan menggunakan prinsip-prinsip OOP dan arsitektur MVC.



Serangan keamanan XSS (Cross-Site Scripting) dan CSRF (Cross-Site Request Forgery) menjadi perhatian utama dalam pengembangan framework MVC berbasis *object* karena memerlukan desain *object* yang kuat pada lingkungan interaksi dan komunikasi antara *users* dan sistem dalam hal mengelola dan memvalidasi input pengguna.

Client Side (Front-end)



Users

Request

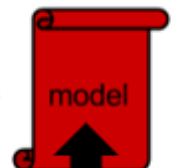
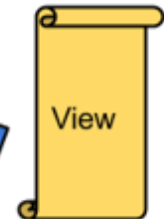
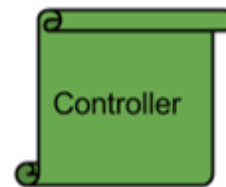
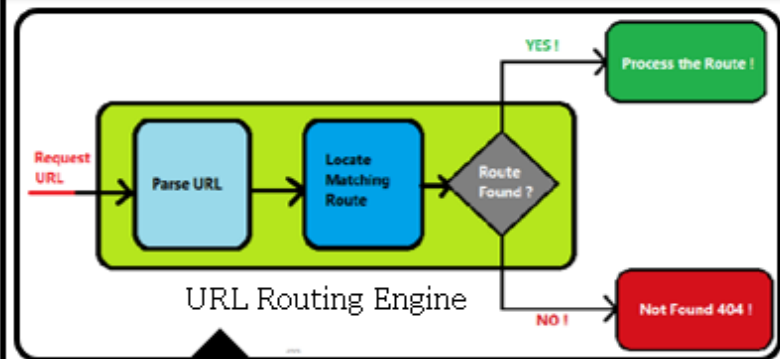


Users

Response



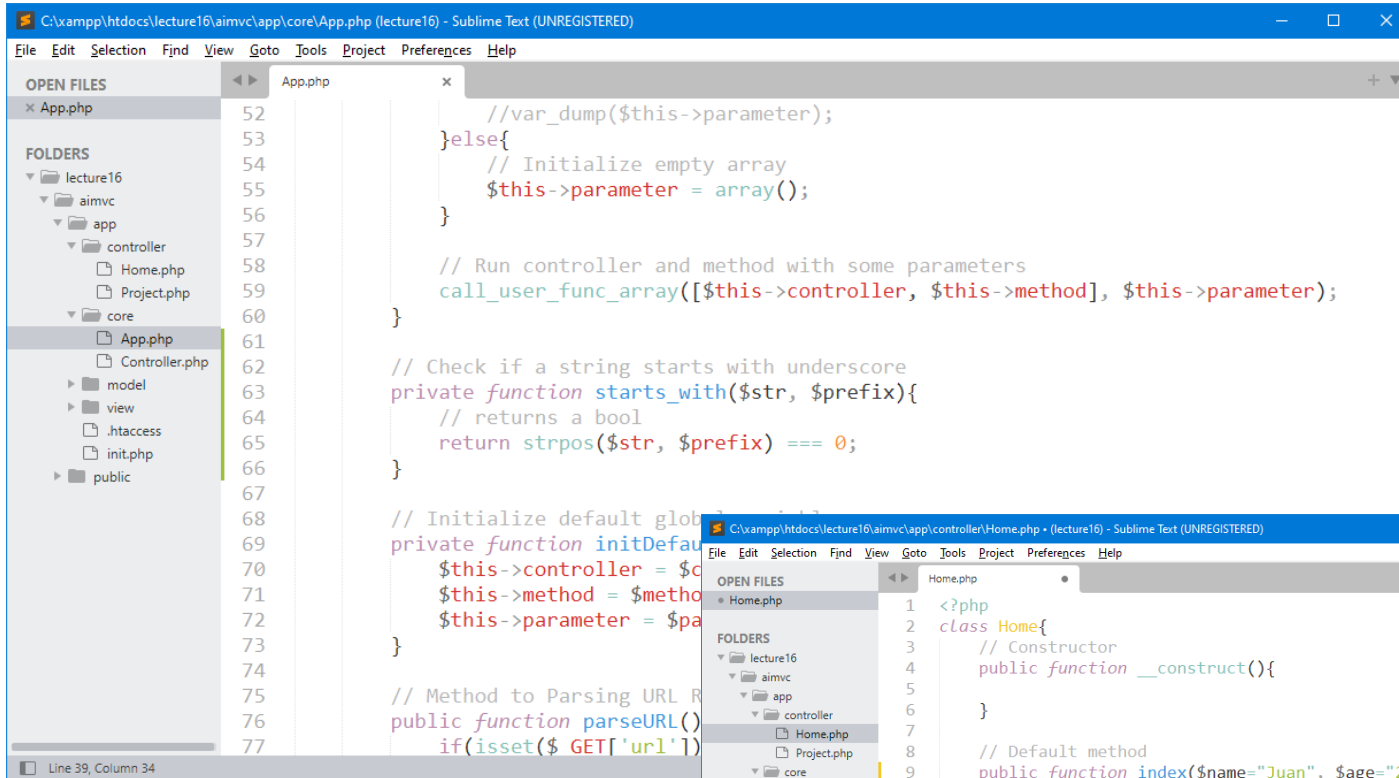
Server Side (Back-end)



Database

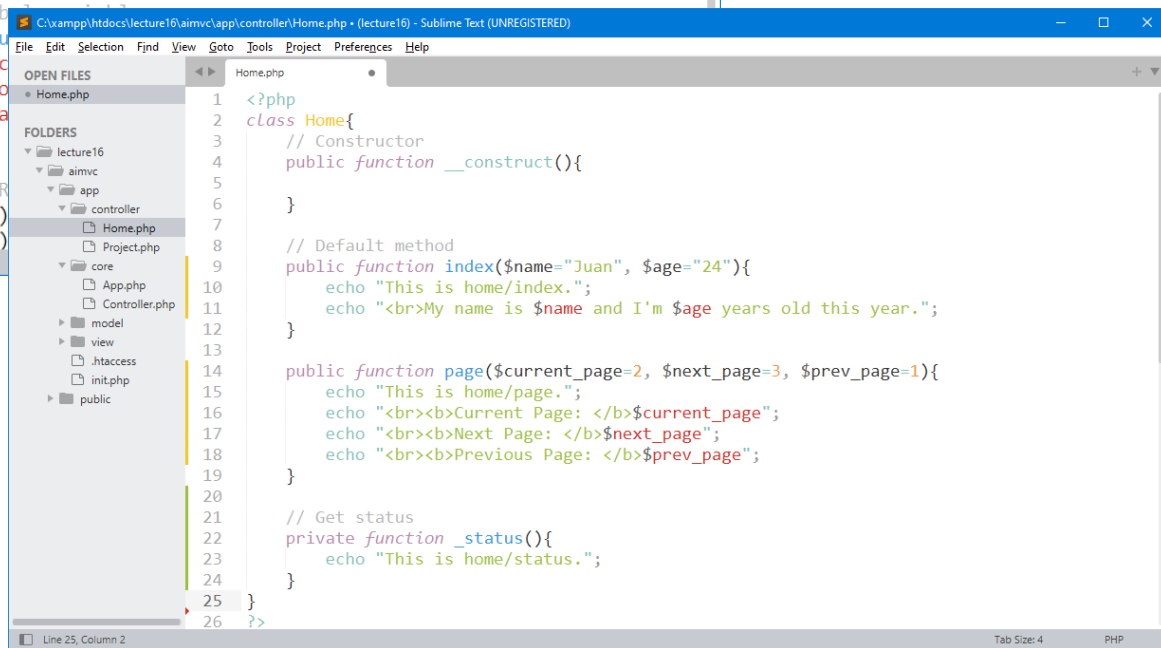


MVC Framework



The screenshot shows the Sublime Text editor with the file `C:\xampp\htdocs\lecture16\aimvc\app\core\App.php` open. The left sidebar displays the project structure under 'lecture16', including folders 'aimvc', 'app', 'controller', 'model', 'view', and files like 'App.php', 'Controller.php', 'init.php', and 'public'. The main editor area shows PHP code for `App.php`, including a `private function starts_with` and a `private function initDefault` method. The status bar at the bottom indicates 'Line 39, Column 34'.

```
52 //var_dump($this->parameter);
53 }else{
54 // Initialize empty array
55 $this->parameter = array();
56 }
57
58 // Run controller and method with some parameters
59 call_user_func_array([$this->controller, $this->method], $this->parameter);
60 }
61
62 // Check if a string starts with underscore
63 private function starts_with($str, $prefix){
64 // returns a bool
65 return strpos($str, $prefix) === 0;
66 }
67
68 // Initialize default global variables
69 private function initDefault(){
70 $this->controller = $this->controller;
71 $this->method = $this->method;
72 $this->parameter = $this->parameter;
73 }
74
75 // Method to Parsing URL Request
76 public function parseURL($url){
77 if(isset($_GET['url'])){
```



The screenshot shows the Sublime Text editor with the file `C:\xampp\htdocs\lecture16\aimvc\app\controller\Home.php` open. The left sidebar displays the project structure under 'lecture16', including folders 'aimvc', 'app', 'controller', 'model', 'view', and files like 'App.php', 'Controller.php', 'init.php', and 'public'. The main editor area shows PHP code for `Home.php`, including a `class Home` with methods `__construct`, `index`, `page`, and `_status`. The status bar at the bottom indicates 'Line 25, Column 2'.

```
1 <?php
2 class Home{
3 // Constructor
4 public function __construct(){
5 }
6
7 // Default method
8 public function index($name="Juan", $age="24"){
9 echo "This is home/index.";
10 echo "<br>My name is $name and I'm $age years old this year.";
11 }
12
13 public function page($current_page=2, $next_page=3, $prev_page=1){
14 echo "This is home/page.";
15 echo "<br><b>Current Page: </b>$current_page";
16 echo "<br><b>Next Page: </b>$next_page";
17 echo "<br><b>Previous Page: </b>$prev_page";
18 }
19
20 // Get status
21 private function _status(){
22 echo "This is home/status.";
23 }
24
25 }
26 ?>
```

Active Class

https://unkpresent.site/unklab/public/dosen/active/

Universitas Klabat

Search

Sign out

Semmy Wellem Taju

GENERAL MENU

Home

Informations

Classroom Topics

Logout

SHORTCUT LECTURER

Active Classes

Archived Classes

Attendance List

Magic Cut

Active Classes

Wednesday, 3 April 202403:14 PM

Reminder! Untuk kesesuaian RPS, diharapkan mengisi topik pembahasan di class pada setiap jadwal presensi.

Course	Meetings/Room	Schedule	Status	Actions
[IS2243] Object Oriented Programming - A 📅 Semester Genap 2023/2024 📅 3 SKS (7 absences max limit) 🏠 Fakultas Ilmu Komputer 👤 Taju, Semmy Wellem	36 Students 📅 21 Created Meetings 📍 GA - 203 Computer Lab	Selasa-Kamis 08.40 - 10.00	active	Options
[IF3262] Back-End Web Development - C 📅 Semester Genap 2023/2024 📅 3 SKS (7 absences max limit) 🏠 Fakultas Ilmu Komputer 👤 Taju, Semmy Wellem	8 Students 📅 23 Created Meetings 📍 GA - 203 Computer Lab	Senin-Rabu 10.10 - 11.30	active	Options
[IF3262] Back-End Web Development - D 📅 Semester Genap 2023/2024 📅 3 SKS (7 absences max limit) 🏠 Fakultas Ilmu Komputer 👤 Taju, Semmy Wellem	27 Students 📅 23 Created Meetings 📍 GA - 203 Computer Lab	Senin-Rabu 14.40 - 16.00	active	Options
[IS2243] Object Oriented Programming - G 📅 Semester Genap 2023/2024 📅 3 SKS (7 absences max limit) 🏠 Fakultas Ilmu Komputer 👤 Taju, Semmy Wellem	33 Students 📅 21 Created Meetings 📍 GK2 Computer Lab 2	Selasa-Kamis 16.10 - 17.30	active	Options
[IS2243] Object Oriented Programming - F 📅 Semester Genap 2023/2024 📅 3 SKS (7 absences max limit)	32 Students 📅 21 Created Meetings	Senin-Rabu 16.10 - 17.30	active	Options

Universitas Klabat

Search

Sign out

Semmy Wellem Taju

GENERAL MENU

Home

Informations

Classroom Topics

Logout

SHORTCUT LECTURER

Active Classes

Archived Classes

Attendance List

Magic Cut

Class Attendance Checklist

Course: [IF4296] DevOps Engineering - A

New +

Summary

Back

	Course Topic	Time	Radius	QR	Actions
	Lecture21 - Collaborative Platform Testing, Managing & Documenting APIs Tuesday, 02 April 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.30 - 16.00	100 Meter	15/18	Options
	Lecture20 - Demo Git Github Jenkins Thursday, 28 March 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.30 - 16.00	100 Meter		
	Lecture19 - Microservices & Infrastructure as Code (IaC) Tuesday, 26 March 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.30 - 16.00	100 Meter	16/18	Options
	Lecture18 - Containerization & Configuration Management Thursday, 21 March 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.00 - 15.00	100 Meter	16/18	Options
	Lecture17 - Demo Source Code Management (SCM) Tuesday, 19 March 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.00 - 15.00	No Distance	17/18	Options
	Lecture16 - Source Code Management & Version Control System (VCS) Thursday, 14 March 2024 Taju, Semmy Wellem semmy@unklab.ac.id	14.30 - 16.00	100 Meter	17/18	Options
	Mid EXAM 2024				

View QR Code

Student Attendance Details

Update Attendance

Delete Attendance

Attendance Summary

https://unkpresent.site/unklab/public/students/summarize/16718

Universitas Klabat

Search

Sign out

Semmy Wellem Taju

GENERAL MENU

Home

Informations

Classroom Topics

Logout

SHORTCUT LECTURER

Active Classes

Archived Classes

Attendance List

Magic Cut

Student Attendance Summary

[IF4296] DevOps Engineering - A

GA - 203 Computer Lab

Go to Classes

Back

No.	Students	Until Today?	Present	Late	Excused	Unexcused	Total
1	Rorong, Giovano ✉ S22110507@student.unklab.ac.id	21 Meetings	18 times	0 time	0 time	3 times	3 absences
2	Adam, Gregorius Vincent ✉ S2200452@student.unklab.ac.id	21 Meetings	20 times	0 time	0 time	1 time	1 absence
3	Barahama, Elvano Rizky ✉ S2200259@student.unklab.ac.id	21 Meetings	20 times	0 time	1 time	0 time	1 absence
4	Besouw, Christian Juan Frederik ✉ S11910217@student.unklab.ac.id	21 Meetings	10 times	5 times	0 time	6 times	7 absences
5	Bogar, Rivaldo ✉ S21910326@student.unklab.ac.id	21 Meetings	14 times	3 times	0 time	4 times	5 absences
6	Dengah, Jofanly Stevanus ✉ S2200066@student.unklab.ac.id	21 Meetings	17 times	1 time	0 time	3 times	3 absences
7	Kalalo, Sherren Brenda ✉ S2200043@student.unklab.ac.id	21 Meetings	18 times	1 time	0 time	2 times	2 absences
8	Katong, Jerico Imanuel ✉ S2200041@student.unklab.ac.id	21 Meetings	21 times	0 time	0 time	0 time	0 absence
9	Kilis, Vannesa Zevania ✉ S2200450@student.unklab.ac.id	21 Meetings	17 times	1 time	1 time	2 times	3 absences

A decorative border of green leaves and branches at the top of the slide.

1 – What is Agregasi (*aggregation*)
in OOP ?

A decorative border of green leaves and branches at the bottom of the slide.

Aggregation *in* OOP

Aggregation

Noun

pengumpulan	collection, accumulation, aggregation, aggregate, solicitation
kesatuan	unity, unit, entity, contingent, corps, aggregation
koleksi	collection, trove, aggregation

Aggregate

Noun

agregat	aggregate, electric generator
keseluruhan	whole, entirety, aggregate, gamut, totality
jumlah	number, amount, total, sum, quantity, aggregate
kumpulan	set, collection, aggregate, body, assembly, bunch
pengumpulan	collection, accumulation, aggregation, aggregate, solicitation muster

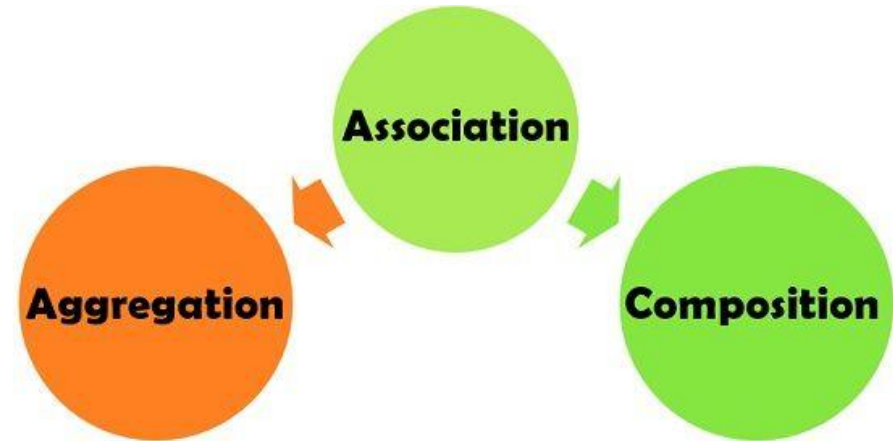
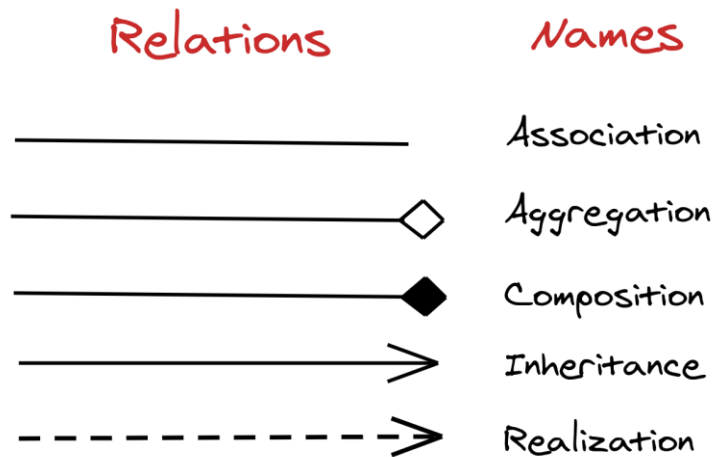
Adjective

yg mengumpulkan	aggregate, accumulative
yg menyatukan	aggregate

Verb

berjumlah	amount, number, total, add up, aggregate, come to
mencapai jumlah	aggregate

Aggregation in OOP



- 1) Aggregation is a specialized form of association between two or more objects in which each object has its own life cycle but there exists an ownership. (*agregasi adalah bentuk asosiasi khusus antara dua atau lebih objek di mana setiap objek memiliki siklus hidupnya sendiri tetapi ada kepemilikan*)
- 2) Aggregation is a typical whole/part or parent/child relationship but it may or may not denote physical containment. (*agregasi adalah hubungan keseluruhan/bagian atau orang tua/anak yang khas tetapi mungkin atau mungkin tidak menunjukkan penahanan fisik*).
- 3) An essential property of an aggregation relationship is that the whole or parent (i.e. the owner) can exist without the part or child and otherwise. (*properti penting dari hubungan agregasi adalah bahwa keseluruhan atau induk (yaitu pemilik) dapat ada tanpa bagian atau anak dan sebaliknya*)

Aggregation OOP *with* Python

Aggregation adalah jenis hubungan di mana:

- 1) Satu objek berfungsi sebagai pemilik objek lainnya, dan
- 2) Objek-objek ini (*lainnya*) membentuk suatu kelompok.
- 3) Objek yang dimiliki tidaklah menjadi milik objek pemilik, dan dapat dimiliki oleh objek lain.

```
# class 1
class Author:
    def __init__(self, name):
        self.name = name
```

Class Author digunakan untuk merepresentasikan penulis buku.

```
# class 2
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
```

Class Book digunakan untuk merepresentasikan sebuah buku.

```
# class 3
class Library:
    def __init__(self, books):
        self.books = books

    def display_books(self):
        print("Books in the library:")
        for book in self.books:
            print("- {} by {}".format(book.title, book.author.name))
```

Class Library digunakan untuk merepresentasikan sebuah perpustakaan yang memiliki koleksi buku.

```
# beberapa object dari kelas Author
author1 = Author("Semmy Taju")
author2 = Author("Eugenie Patricia")
author3 = Author("Stevi Gabriel")

# beberapa object dari kelas Book
book1 = Book("Java Programming", author1)
book2 = Book("Python Programming", author2)
book3 = Book("Python Programming", author3)

# object dari kelas Library dan menambahkan buku-buku ke dalam object
library = Library([book1, book2, book3])

# call metode display_books untuk menampilkan buku-buku di dalam perpustakaan
library.display_books()
```

Di dalam kelas Library, terdapat atribut books yang merepresentasikan koleksi buku di dalam perpustakaan. Atribut ini merupakan contoh dari **hubungan Composition or Aggregation???**, karena buku-buku di dalam perpustakaan (**tidak sepenuhnya or sepenuhnya**) bergantung pada perpustakaan untuk bertahan hidup.

Aggregation OOP *with* Python

Aggregation adalah jenis hubungan di mana:

- 1) Satu objek berfungsi sebagai pemilik objek lainnya, dan
- 2) Objek-objek ini (*lainnya*) membentuk suatu kelompok.
- 3) Objek yang dimiliki tidaklah menjadi milik objek pemilik, dan dapat dimiliki oleh objek lain.

```
1 # class 1
2 class Mobil:
3     def __init__(self, merk, tahun):
4         self.merk = merk
5         self.tahun = tahun
6         self.pemilik = None
7
8     def display(self):
9         print("Mobil:", self.merk, "Tahun:", self.tahun)
10        if self.pemilik:
11            self.pemilik.display()
12
13 # class 2
14 class Pemilik:
15     def __init__(self, nama, alamat):
16         self.nama = nama
17         self.alamat = alamat
18
19     def display(self):
20         print("Pemilik:", self.nama, "Alamat:", self.alamat)
21
22 # Membuat objek Pemilik
23 pemilik1 = Pemilik("Semmy Taja", "Jl. yordan No. 123")
24
25 # Membuat objek Mobil
26 mobil1 = Mobil("Toyota Avanza", 2023)
27
28 # Menghubungkan objek Mobil dengan objek Pemilik
29 mobil1.pemilik = pemilik1
30
31 # Menampilkan informasi mobil beserta pemiliknya
32 mobil1.display()
```

Pada aggregation, objek yang satu bisa digunakan di objek yang lain tanpa menjadi bagian dari objek tersebut.

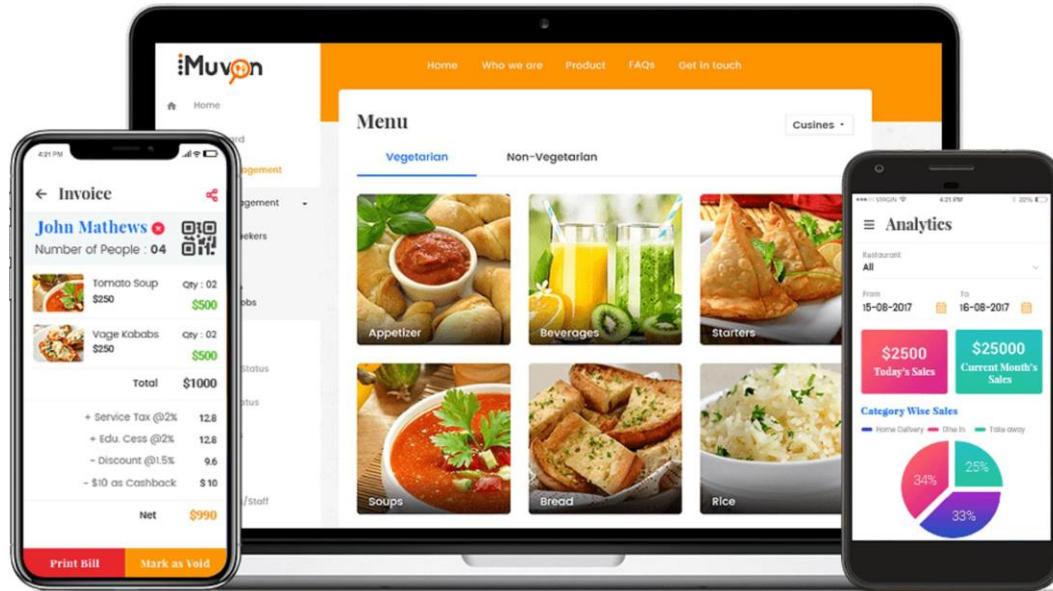
Artinya, **objek yang satu tidak tergantung pada objek yang lain dan dapat hidup secara mandiri.**

kita menghubungkan antara objek *mobil1* dengan objek *pemilik1* dengan melakukan assignment *mobil1.pemilik = pemilik1*.

Jika object *mobil1* di hapus maka object *pemilik1* tidak terganggu dan dapat hidup mandiri.



Exercise *for* Students



Restaurant Management System

McDonald Resto *with* OOP Python

```
# class 1
class Restaurant:
    # constructor without parameters
    def __init__(self):
        self.menu = {
            "Nasi Goreng": 15000,
            "Mie Goreng": 12000,
            "Nasi Padang": 25000,
            "Ayam Goreng": 18000,
            "Soto Ayam": 20000
        }
        self.order_list = []
        self.total_price = 0

    # fungsi menampilkan menu restoran
    def show_menu(self):
        print("Menu:")
        for item in self.menu:
            print("(Rp. {0}) {1}".format(self.menu[item], item))

    # fungsi order menu restoran
    def order(self, item, qty):
        if item in self.menu:
            price = self.menu[item] * qty
            self.order_list.append((item, qty, price))
            self.total_price += price
            print(qty, item, "added to order.")
        else:
            print(item, "not found in menu.")

    # fungsi menghandle proses pembayaran
    def buy(self):
        if len(self.order_list) > 0:
            print("Order Summary:")
            for order in self.order_list:
                print(order[1], order[0], "[ Rp. ", order[2], "]")
            print("Total Price:", "Rp. ", self.total_price)
            self.order_list = []
            self.total_price = 0
        else:
            print("No order placed.")
```

1

```
# class 2
class McDonaldRestaurant(Restaurant):
    # constructor with parameters
    def __init__(self):
        super().__init__() # call parent class constructor
        self.menu.update({
            "Burger": 20000,
            "Hotdog": 15000,
            "French Fries": 10000,
            "Fried Chicken": 25000,
            "Pizza": 30000
        })
```

2

Inheritance
atau pewarisan
dalam OOP.

```
# create object
restaurant = McDonaldRestaurant()

# loop while untuk menampilkan menu
while True:
    print("\nWelcome to Semmy Restaurant:")
    print("1. Show Menu")
    print("2. Order")
    print("3. Buy")
    print("4. Exit")

    # switch case (if-elif-else statement)
    choice = int(input("Enter menu number: "))
    # menu 1
    if choice == 1:
        restaurant.show_menu()
    # menu 2
    elif choice == 2:
        selected_item = input("Enter item name (mau pesan apa): ")
        num_item = int(input("Enter quantity (berapa banyak): "))
        restaurant.order(selected_item, num_item)
    # menu 3
    elif choice == 3:
        restaurant.buy()
    # menu 4
    elif choice == 4:
        break
    else:
        print("Sorry, the number is not in the menu.")
```

3

Expected Output

Welcome to Semmy Restaurant:

1. Show Menu
2. Order
3. Buy
4. Exit

Enter menu number: 1

Menu:

- (Rp. 15000) Nasi Goreng
- (Rp. 12000) Mie Goreng
- (Rp. 25000) Nasi Padang
- (Rp. 18000) Ayam Goreng
- (Rp. 20000) Soto Ayam
- (Rp. 20000) Burger
- (Rp. 15000) Hotdog
- (Rp. 10000) French Fries
- (Rp. 25000) Fried Chicken
- (Rp. 30000) Pizza

Welcome to Semmy Restaurant:

1. Show Menu
2. Order
3. Buy
4. Exit

Enter menu number: 8

Sorry, the number is not in the menu.

Welcome to Semmy Restaurant:

1. Show Menu
2. Order
3. Buy
4. Exit

Enter menu number: 2

Enter item name (mau pesan apa): Soto Ayam

Enter quantity (berapa banyak): 3
3 Soto Ayam added to order.

Welcome to Semmy Restaurant:

1. Show Menu
2. Order
3. Buy
4. Exit

Enter menu number: 3

Order Summary:

3 Soto Ayam [Rp. 60000]
Total Price: Rp. 60000

Welcome to Semmy Restaurant:

1. Show Menu
2. Order
3. Buy
4. Exit

Tidak ada **BATASAN** fungsi dalam pengembangan Restaurant Management System (RMS) berbasis OOP sederhana ini, kalian bisa berkreasi dengan menambahkan fungsi ataupun fitur-fitur yang baru. Buatlah versi restaurant kalian sendiri.

Selain dalam bentuk REPORT, students juga akan diminta untuk menjelaskan ALUR PROSES setiap METHOD, output dan demo program !!!

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

